

ECE 124 – Digital Circuits and Systems
Dept. of ECE, Univ. of Waterloo

Lecture Slides: **Set 8 - Part 1**

Contents:

- State reduction in asynchronous sequential circuits

©2014-2026 M. A. Hasan. These slides and notes are for the exclusive use of the students registered in the course. Reproduction in any form or use for any other purposes is prohibited.

State reduction

Our state reduction task involves two procedures

1. Partitioning of *equivalent* states
2. Merging of *compatible* states

Partitioning

- Equivalent states are identified and grouped together
- Conditions for equivalent states were discussed in Slide set 7
- We will impose an additional constraint— two states, i.e., rows, in a flow table to be potentially equivalent any unspecified entries must be in the same NS column
- Thus, combining two equivalent states into a single state will not remove the don't cares
- (In merging, rows are merged taking advantage of unspecified entries)

Partitioning

Consider the following initial flow table. (Note that each row has only one stable state. Such a flow table is called primitive flow table)

Present State	Next state					Output z
	uv	$= 00$	01	10	11	
A		(A)	B	C	—	0
B		D	(B)	—	—	0
C		A	—	(C)	—	1
D		(D)	E	F	—	0
E		A	(E)	—	—	1
F		A	—	(F)	—	1

Initial flow table

After partitioning based on output
 $P_0 = (ABD)(CEF)$

Because of different NS
 $(ABD) \rightarrow (A)(B)(D)$

Rows with unspecified entries can be in the same partition if their unspecified entries are aligned. Thus
 $(CEF) \rightarrow (CF)(E)$

Combining the above partitions
 $P_1 = (A)(B)(D)(CF)(E)$

Partitioning (contd.)

Present state	Next state					Output z	
	uv	$=$	00	01	10		11
A			(A)	B	C	—	0
B			D	(B)	—	—	0
C			A	—	(C)	—	1
D			(D)	E	C	—	0
E			A	(E)	—	—	1

Modified flow table after first-step reduction

Merging

Compatible states: Two states (rows in a flow table) S_i and S_j are said to be *compatible* if there are no state conflicts for any input valuation. In other words, S_i and S_j are compatible if the following two hold

1. both S_i and S_j must have the same output whenever specified, and
2. for *each* input valuation, one of the following conditions must be true
 - a. both S_i and S_j have the same successor, or
 - b. both S_i and S_j are stable, or
 - c. The successor of S_i or S_j , or both, is unspecified

Merging (contd.)

Steps for merging:

1. Identify compatible state pairs
2. Draw a merger diagram showing relationship among various states, where
 - Each row of the flow table is represented as a point, labeled by the row name
 - A line is drawn connecting any two points that correspond to compatible states (rows)
3. From the merger diagram, choose the best merging possibility. (The goal is to have as few states as possible)

Note that for any set of rows that are pairwise compatible, all pairs in the set can be merged into a single state

Example for merging

Assume that the following (primitive) flow table is given

Present state	Next state				Output z
	$w_2 w_1 =$ 00	01	10	11	
A	(A)	H	B	—	0
B	F	—	(B)	C	0
C	—	H	—	(C)	1
D	A	(D)	—	E	1
E	—	D	G	(E)	1
F	(F)	D	—	—	0
G	F	—	(G)	—	0
H	—	(H)	—	E	0

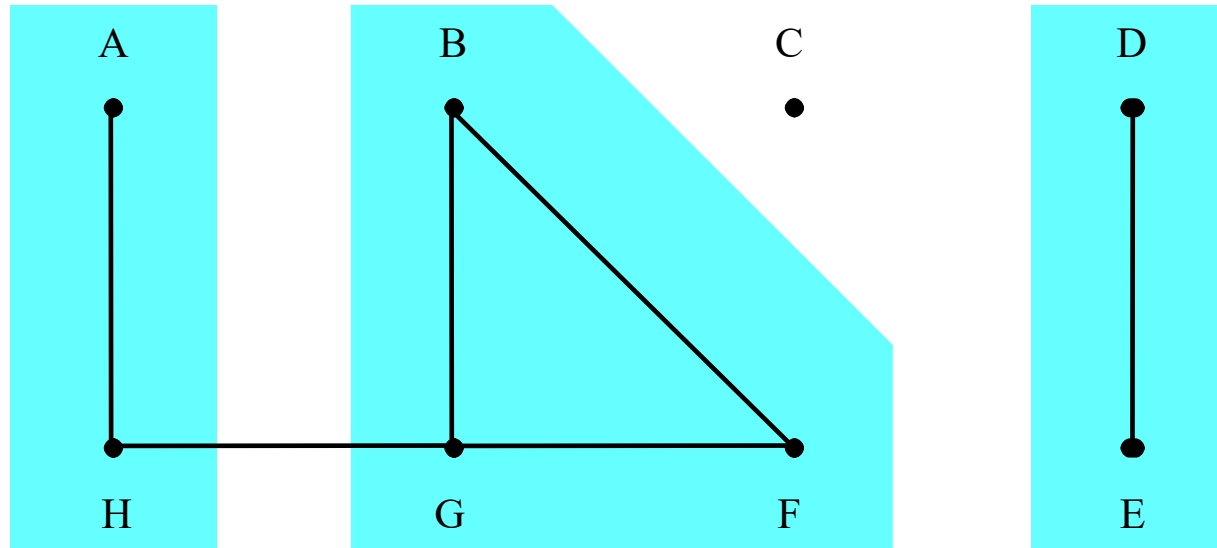
Compatible pair(s) with $z=0$:

- (A, H)
- (B, F), (B, G)
- (F, G)
- (G, H)

Compatible pair(s) with $z=1$:

- (D, E)

Example for merging (contd.)



Notes:

- A and H can be merged as long as H is not merged with G
- B, G and F can be merged to a single state (row)
- D and E can be merged

Example for merging (contd.)

After merging, the following reduced flow table is obtained (the total number of states has gone down to 4 from 8)

Present state	Next state				Output z
	$w_2 w_1 = 00$	01	10	11	
A	(A)	(A)	B	D	0
B	(B)	D	(B)	C	0
C	—	A	—	(C)	1
D	A	(D)	B	(D)	1

State reduction procedure

Steps:

1. Use the partitioning procedure to eliminate all but one state for each set of equivalent states
2. Construct a merger diagram
3. Chose a subset of compatible states that can be merged into a single state
 - Try to minimize the # of subsets needed to cover all states
 - Each state must be included in only one of the chosen subsets
4. Derive the reduced flow table by merging the rows in chosen subsets
5. Repeat steps 2 to 4 to see whether further reductions are possible

ECE 124 – Digital Circuits and Systems
Dept. of ECE, Univ. of Waterloo

Lecture Slides: **Set 8 - Part 2**

Contents:

- Example of state reduction

©2014-2026 M. A. Hasan. These slides and notes are for the exclusive use of the students registered in the course. Reproduction in any form or use for any other purposes is prohibited.

Example for state reduction procedure

Assume that the flow table at right is given.

Partitioning based on output

$$P_0 = (ACFGJ)(BDEHKL)$$

Considering NS/successors & unspecified entries:

$$(ACFGJ) \rightarrow (A)(C)(F)(G)(J)$$

$$(BDEHKL) \rightarrow (BL)(D)(E)(HK)$$

Note: you may find it easier to rewrite rows based on their outputs and scanned them in sequence for partitioning, e.g., first A, then C, then F, etc. for $z=0$.

Present state	Next state				Output z
	$w_2 w_1 = 00$	01	10	11	
A	(A)	F	C	—	0
B	A	(B)	—	H	1
C	G	—	(C)	D	0
D	—	F	—	(D)	1
E	G	—	(E)	D	1
F	—	(F)	—	K	0
G	(G)	B	J	—	0
H	—	L	E	(H)	1
J	G	—	(J)	—	0
K	—	B	E	(K)	1
L	A	(L)	—	K	1

Example for state reduction procedure (contd.)

Reduction obtained by using the partitioning procedure from the previous slide

Present state	Next state				Output z
	$w_2 w_1 = 00$	01	10	11	
A	<u>A</u>	F	C	—	0
B	A	<u>B</u>	—	H	1
C	G	—	<u>C</u>	D	0
D	—	F	—	<u>D</u>	1
E	G	—	<u>E</u>	D	1
F	—	<u>F</u>	—	H	0
G	<u>G</u>	B	J	—	0
H	—	B	E	<u>H</u>	1
J	G	—	<u>J</u>	—	0

Compatible pair(s) for $z=0$:

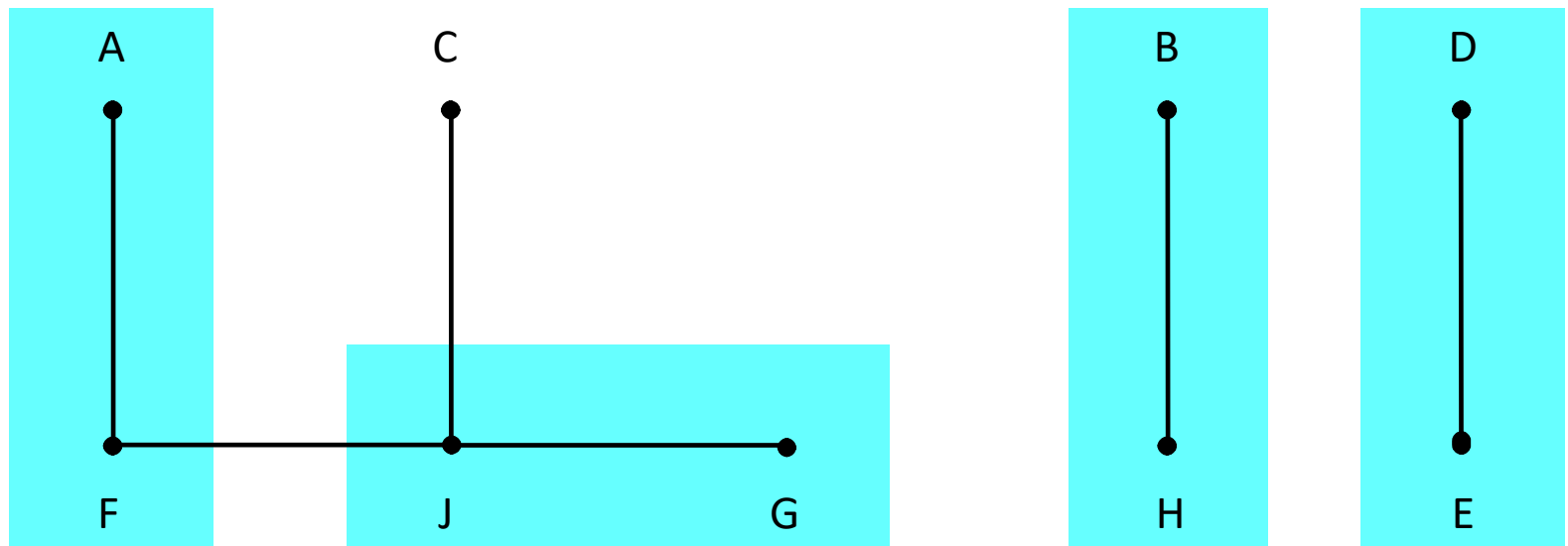
- (A, F)
- (C, J)
- (F, J)
- (G, J)

Compatible pair(s) for $z=1$:

- (B, H)
- (D, E)

Example for state reduction procedure (contd.)

Merger diagram is shown below along with connections among various points based on the flow table and compatible pairs identified in the previous slide). The resulting flow table is in the next slide.



Example for state reduction procedure (contd.)

Reduced flow table (after merging shown in the previous slide)

Present state	Next state				Output z
	$w_2 w_1 = 00$	01	10	11	
A	$\textcircled{A}$	$\textcircled{A}$	C	B	0
B	A	$\textcircled{B}$	D	$\textcircled{B}$	1
C	G	—	$\textcircled{C}$	D	0
D	G	A	$\textcircled{D}$	$\textcircled{D}$	1
G	$\textcircled{G}$	B	$\textcircled{G}$	—	0

Merger diagram for the above flow table:

A



B



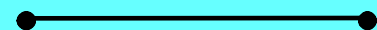
D



C



G



Example for state reduction procedure (contd.)

Below is the reduced flow table based on the merger diagram from the bottom of the previous slide. (No more merging can be done for this example)

Present state	Next state				Output z
	$w_2 w_1 = 00$	01	10	11	
A	A	A	C	B	0
B	A	B	D	B	1
C	C	B	C	D	0
D	C	A	D	D	1

ECE 124 – Digital Circuits and Systems
Dept. of ECE, Univ. of Waterloo

Lecture Slides: **Set 8 - Part 3**

Contents:

- Race free state assignment

©2014-2026 M. A. Hasan. These slides and notes are for the exclusive use of the students registered in the course. Reproduction in any form or use for any other purposes is prohibited.

Race conditions

- A race condition occurs in ASC when two or more state variables change in response to a change in the circuit input value
- **Unequal circuit delays** may not allow multiple state variables change at the same time
- Assume that two state variables change.
 - If the circuit reaches the **same** final, stable state regardless of the order in which the state variables change, then the race is **non-critical**
 - If the circuit reaches a final, stable state **depending on the order** in which the state variables change, then the race is **critical**
- We need to avoid critical races for predictability and to ensure our circuit does the intended function

Race conditions (contd.)

Notes:

- Races are a consequence of how states are assigned when designing a circuit (races do not exist in flow tables, where states are represented as symbols)
- We can prevent races to occur by performing state assignment such that transitions from one stable state to another state require only one state variable to change at a time
- A number of methods are available for race free state assignment

Method # 1

This method is based on the use of **extra unstable** states.

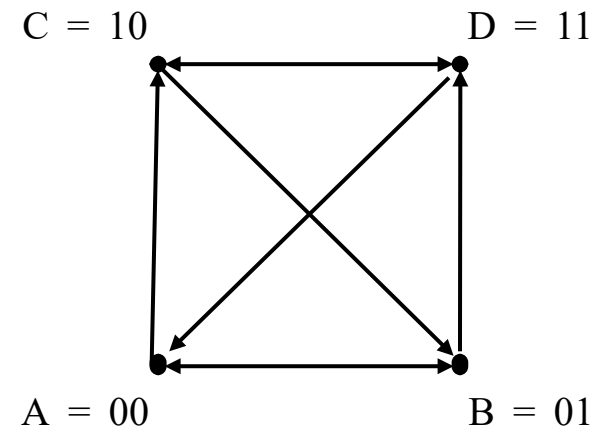
Let there be N states in the flow table and $n = \lceil \log_2 N \rceil$.

- Try to embed the symbolic states of the flow table into the co-ordinates of the n or a **higher** dimensional 'cube' such that the path from stable state to stable state:
 - is direct along a single edge of the cube, or
 - goes through newly introduced unstable states along cube edges
- New extra states are always **unstable**

Method # 1 (contd.)

Example: Consider the following flow table

Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 = 00$	01	10	11	
A	(A)	(A)	C	B	00
B	A	(B)	D	(B)	01
C	(C)	B	(C)	D	10
D	C	A	(D)	(D)	11



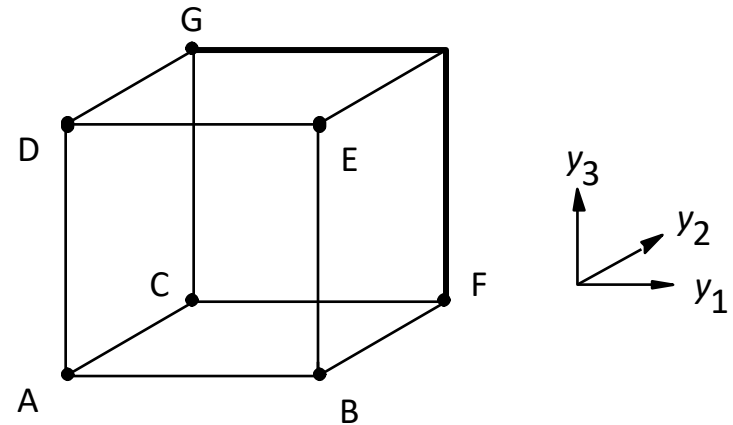
Here $n = 2$ and we first try a 2-dimensional 'cube' (i.e., a square).

- A possible transition diagram is shown above (right)
- Each **diagonal** represents value changes in multiple (two) state variables (we need to avoid diagonals)
- Try other state assignments on the square, but you will not be able to get rid of diagonals for this example

Method # 1 (contd.)

Next, we try the 3-D cube (see below)

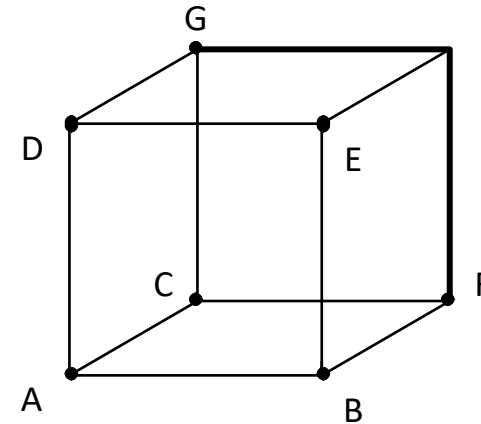
Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 = 00$	01	10	11	
A	(A)	(A)	C	B	00
B	A	(B)	D	(B)	01
C	(C)	B	(C)	D	10
D	C	A	(D)	(D)	11



- Extra co-ordinates in the 3-D cube allow us to
 - avoid diagonals by placing more states adjacent to each other and
 - introduce additional, **unstable** states and use them as intermediate states during transitions.
- For example, in the above 3-D cube
 - D is placed adjacent to A (i.e., the D-to-A transition requires the value of only one state variable change)
 - Transition from B to D is via intermediate state E and, similarly, from C to B is via F, each requiring a change in one variable only.
- All transitions are shown in the table in the next slide.

Method # 1 (contd.)

Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 = 00$	01	10	11	
A	(A)	(A)	C	B	00
B	A	(B)	D	(B)	01
C	(C)	B	(C)	D	10
D	C	A	(D)	(D)	11



Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 = 00$	01	10	11	
A	(A)	(A)	C	B	00
B	A	(B)	E	(B)	01
C	(C)	F	(C)	G	10
D	G	A	(D)	(D)	11
E	—	—	D	—	??
F	—	B	—	—	??
G	C	—	—	D	??

- In the new table, states are represented as $y_3 y_2 y_1$, i.e., A=000, B=001, C=010, D=100, E=101, F=011 and G=110 (see the 3-D cube in the previous slide)
- This state assignment has removed all race conditions.

Method # 1 (contd.)

Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 =$ 00	01	10	11	
A	A	A	C	B	00
B	A	B	E	B	01
C	C	F	C	G	10
D	G	A	D	D	11
E	—	—	D	—	—1
F	—	B	—	—	01
G	C	—	—	D	1—

- To reduce unnecessary delay, the output of F is set to that of the 'effective' next state B (rather than that of the 'effective' present state C).
- In addition, '—' is not used for the output of F. Rationale: during optimization of the circuit for $z_2 z_1$, entry '—' could be retreated as '11' or '00', neither of which corresponds to the output of the effective present or next state.

Method # 2

- This method is useful for a flow table with ≤ 4 states.
- It is based on the use of **equivalent states** , i.e., each state of the given flow table is replaced with two (or more) equivalent states
- E.g., state A can be replaced by two equivalent states A1 and A2.
- The equivalent states A1 and A2 have the following features
 - output for A1 and A2 should be the same as that for A
 - if the original state A is stable for an input valuation, then its equivalent states A1 and A2 are also stable for the same input valuation
 - for an input valuation, the (ultimate) stable successors for A1 and A2 belong to the same equivalent pair

Method # 2 (contd.)

- For a given flow table of N states, this method creates $2N$ states (because of 2 equivalent states of each original state)
- Embedding rule:

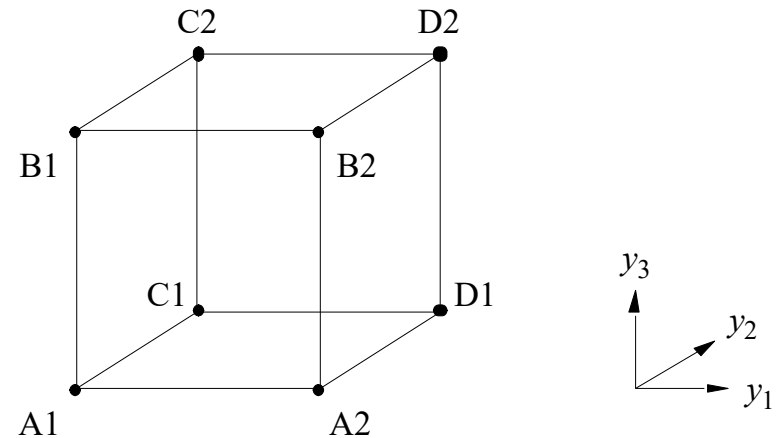
The $2N$ states are embedded into a cube of dimension $n + 1$ or higher such that

- members (e.g., states A1 and A2) of an equivalent pair are adjacent to each other
- Through members, each pair is adjacent to other pairs

Method # 2 (contd.)

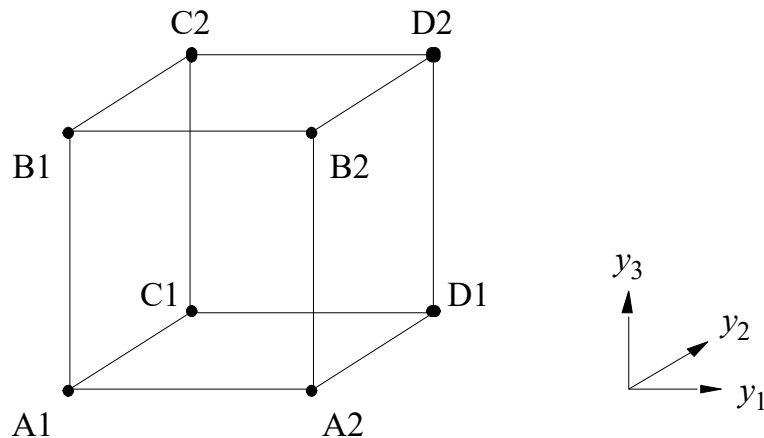
Example: Consider the following flow table (same as before)

Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 =$ 00	01	10	11	
A	(A)	(A)	C	B	00
B	A	(B)	D	(B)	01
C	(C)	B	(C)	D	10
D	C	A	(D)	(D)	11



One possible embedding of the equivalent states in the 3-D cube is shown above. The corresponding modified flow table is shown in the next slide.

Method # 2 (contd.)



- The cube is repeated from previous slide
- In the cube, A1=000, A2=001, B1=100, B2=101, C1=010, C2=110, D1=011 and D2=111

Present state	Next state				Output $z_2 z_1$
	$w_2 w_1 = 00$	01	10	11	
A1	(A1)	(A1)	C1	B1	00
A2	(A2)	(A2)	A1	B2	00
B1	A1	(B1)	B2	(B1)	01
B2	A2	(B2)	D2	(B2)	01
C1	(C1)	C2	(C1)	D1	10
C2	(C2)	B1	(C2)	D2	10
D1	C1	A2	(D1)	(D1)	11
D2	C2	D1	(D2)	(D2)	11

Determination of transitional next states, i.e., states not circled in the above table. For example:

- For input 10, A1 needs to transit to C1 or C2. Based on how we have embedded the states in the above cube, C1 (not C2) is adjacent to A1; so A1 transits to C1.
- For input 10, A2 needs to transit to C1 or C2. However, C1 and C2 are not adjacent to A2. In this situation, A2 transits to its equivalent state A1, which by the embedding rule must be adjacent to either C1 or C2 (in this case C1)

Method # 3

- This method is based on the use of **one-hot-encoding**
- For a given flow table, let the i -th state, $1 \leq i \leq N$, be represented as an N -bit binary number where the i -th bit is 1 and all other bits are zero. (E.g., for $N=4$, if the states are $S_4 \dots S_1$, then S_2 is encoded as 0010)
- For a transition from i -th stable state to j -th stable state, introduce unstable state with encoding $0\dots010 \dots 010\dots0$, where 1s are in the i -th and j -th positions only.
- Consider the original flow table shown with Method # 2. Create a modified table by looking at transitions and introducing new unstable states as required using one-hot-encoding (see the next slide).

Method # 3 (contd.)

Present state	Next state				Output z_2z_1
	$w_2w_1 =$ 00	01	10	11	
A	(A)	(A)	C	B	00
B	A	(B)	D	(B)	01
C	(C)	B	(C)	D	10
D	C	A	(D)	(D)	11

State assignment	Present State	Nextstate				Output z_2z_1	
		$w_2w_1 =$	00	01	10		11
0001	A		(A)	(A)	E	F	00
0010	B		F	(B)	G	(B)	01
0100	C		(C)	H	(C)	I	10
1000	D		I	J	(D)	(D)	11
0101	E		—	—	C	—	—0
0011	F		A	—	—	B	0—
1010	G		—	—	D	—	—1
0110	H		—	B	—	—	01
1100	I		C	—	—	D	1—
1001	J		—	A	—	—	00

ECE 124 – Digital Circuits and Systems
Dept. of ECE, Univ. of Waterloo

Lecture Slides: **Set 8 - Part 4**

Contents:

- Hazards free circuits

©2014-2026 M. A. Hasan. These slides and notes are for the exclusive use of the students registered in the course. Reproduction in any form or use for any other purposes is prohibited.

Introduction to hazards

- A **hazard** is a momentary **unwanted switching transient** at a circuit's output (i.e., glitch)
- Hazards/glitches occur due to unequal propagation delays along different paths in a combinational circuit
- For asynchronous sequential circuits (ASC), hazards can cause problems. For example, momentary false logic values in an ASC can cause a transition to an incorrect stable state
- Hazards can be static or dynamic

Introduction to hazards (contd.)

- Static-1 hazard: it occurs when output is 1 and should remain at 1, but temporarily switches to 0 due to a change in an input
- Static-0 hazard: it occurs when output is 0 and should remain at 0, but temporarily switches to 1 due to a change in an input



Introduction to hazards (contd.)

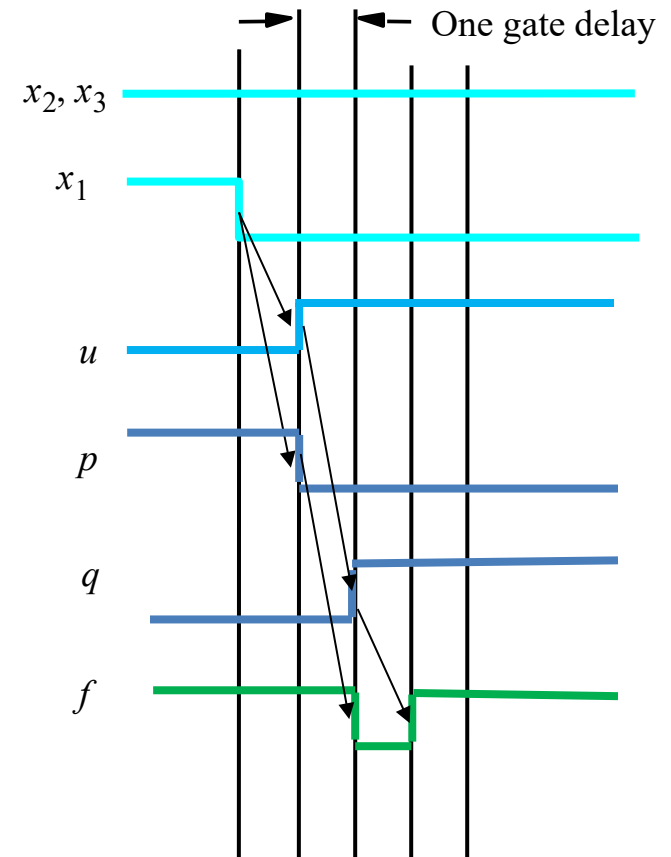
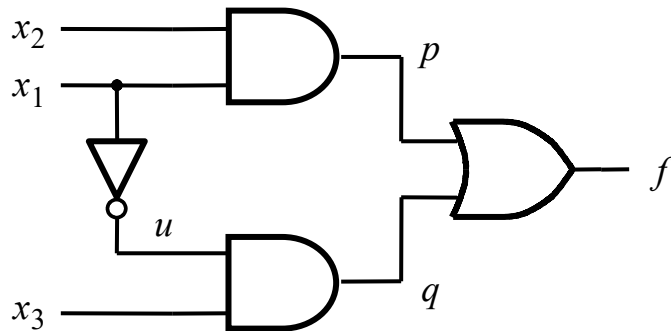
- Dynamic hazard: It occurs when an input changes, and a circuit output should change 0 → 1 or 1 → 0, but temporarily flips between values



Dynamic hazard

Static hazards (in SOP circuits)

- Consider the following circuit.
- Assume that all three inputs of the circuit are initially 1. For this set of input values, we have output $f = 1$.
- Now let us change input x_1 from 1 to 0 and follow the timing diagram assuming that all gates have the same propagation delay

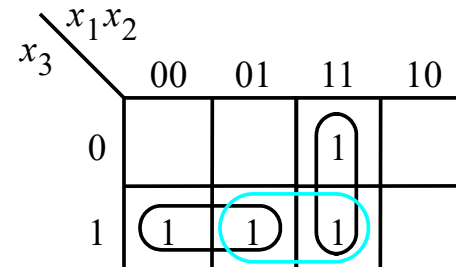
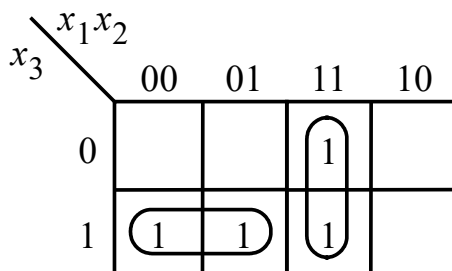


Timing diagram

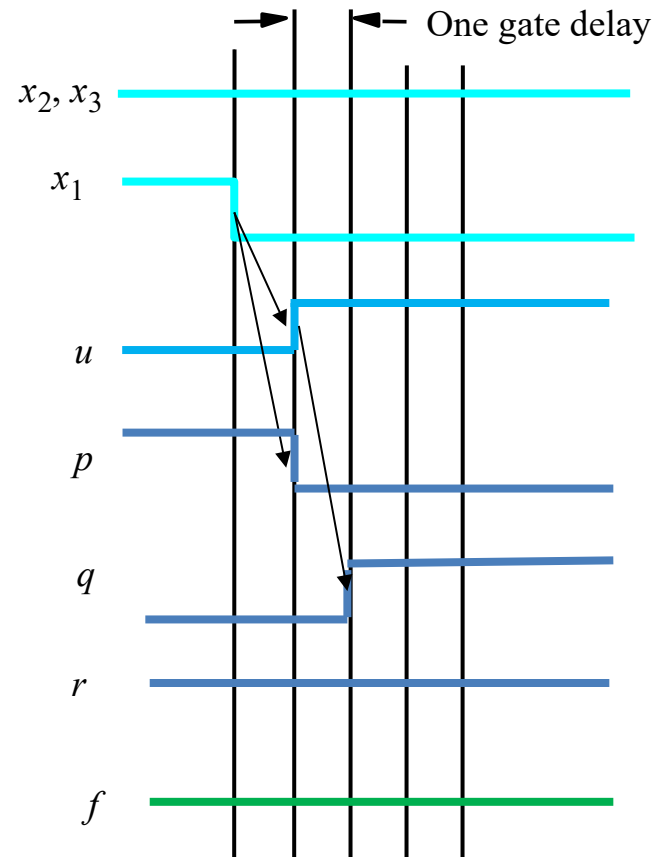
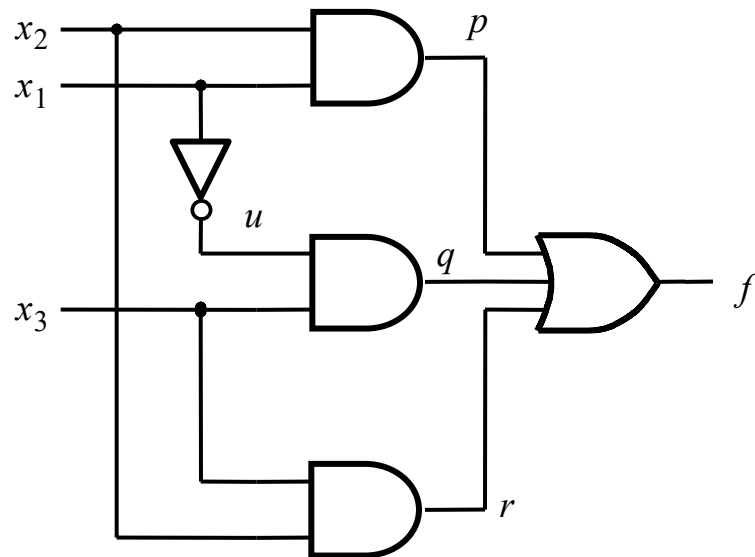
- Based on the logic expression $f = x_1x_2 + x_1'x_3$, we expect the output to remain at logic 1, but due to delay, the output goes to 1->0->1, i.e., we have a glitch at the output. This is a static-1 hazard.

Hazard free SOP circuits

- When circuits are implemented as a 2-level SOP (resp. 2-level POS), we can detect and remove hazards by inspecting the K-map and adding redundant product (resp. sum) terms.
- The K-map for the function of the previous slide is shown below (left).
- When x_1 changes from 1 to 0, the function value jumps from one product term to another
- When adjacent minterms are not covered by the same product term, then a hazard exists
- To cover adjacent minterms and avoid hazards, we can include extra product terms in the logic expression for the output
- The extra product term in the K-map below right does not include the changing input x_1 preventing possible momentary output glitches. (The corresponding circuit and timing diagram are in the next slide)



Hazard free SOP circuits (contd.)

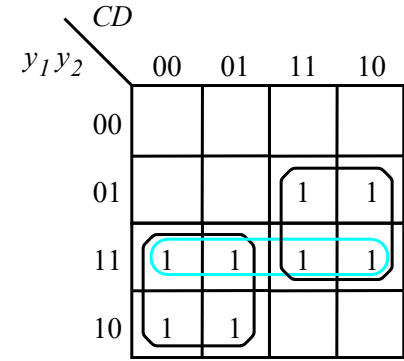
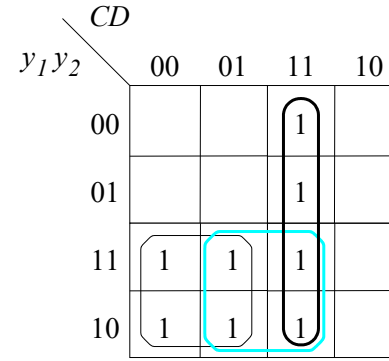


Timing diagram

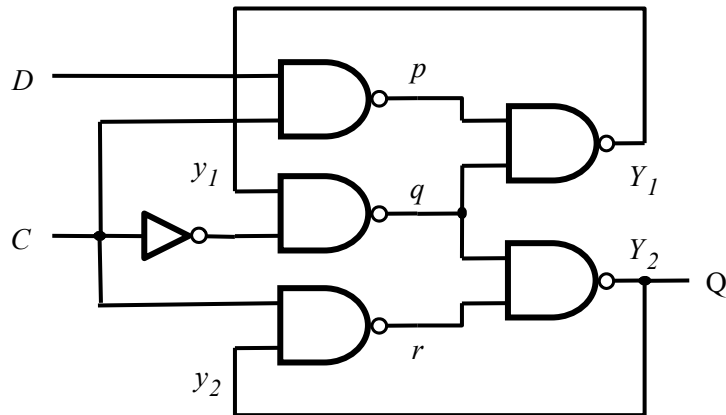
Hazard free SOP circuits (contd.)

Present state y_1y_2	Next state				Output Q
	$CD = 00$	01	10	11	
	Y_1Y_2				
00	00	00	00	10	0
01	00	00	01	11	1
10	11	11	00	10	0
11	11	11	01	11	1

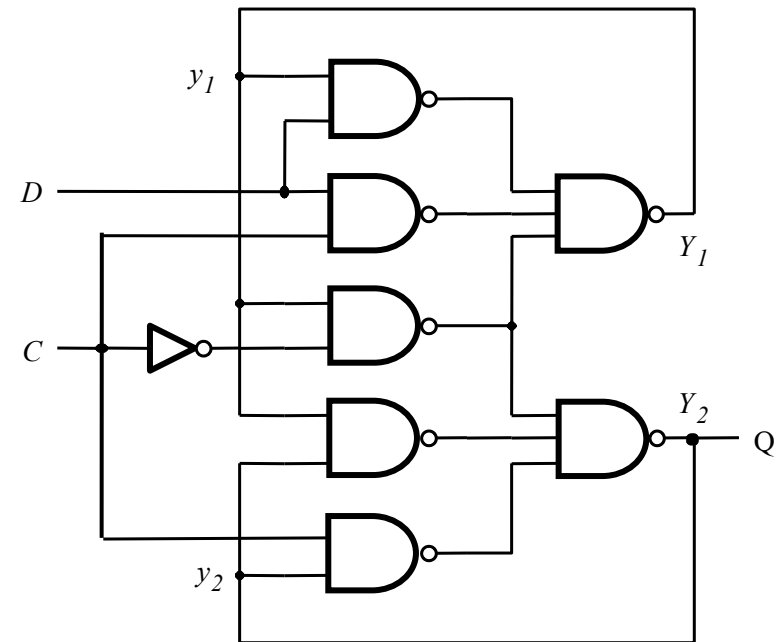
(a) An example excitation table



(b) Karnaugh maps for Y_1 and Y_2

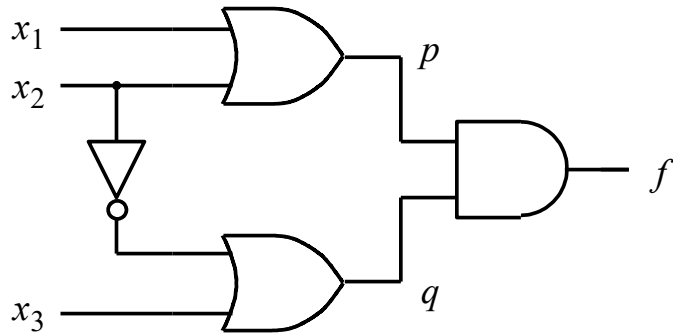


(c) Minimum-cost circuit

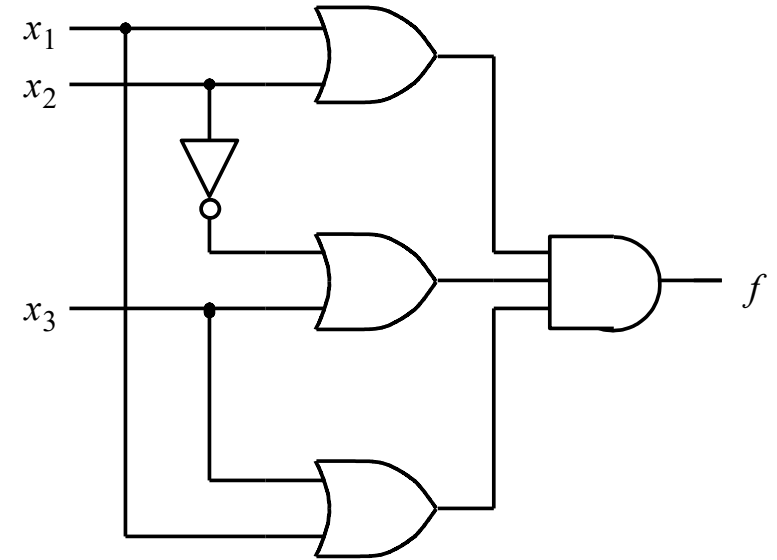


(d) Hazard-free circuit

Hazard free POS circuits



(a) Circuit with a hazard



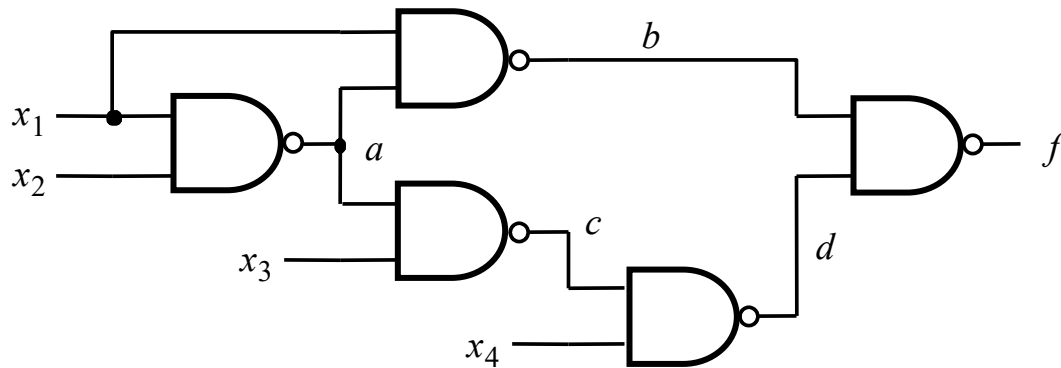
(c) Hazard-free circuit

$x_1 x_2$		00	01	11	10
x_3	0	0	0	0	1
	1	0	1	1	1

(b) Karnaugh map

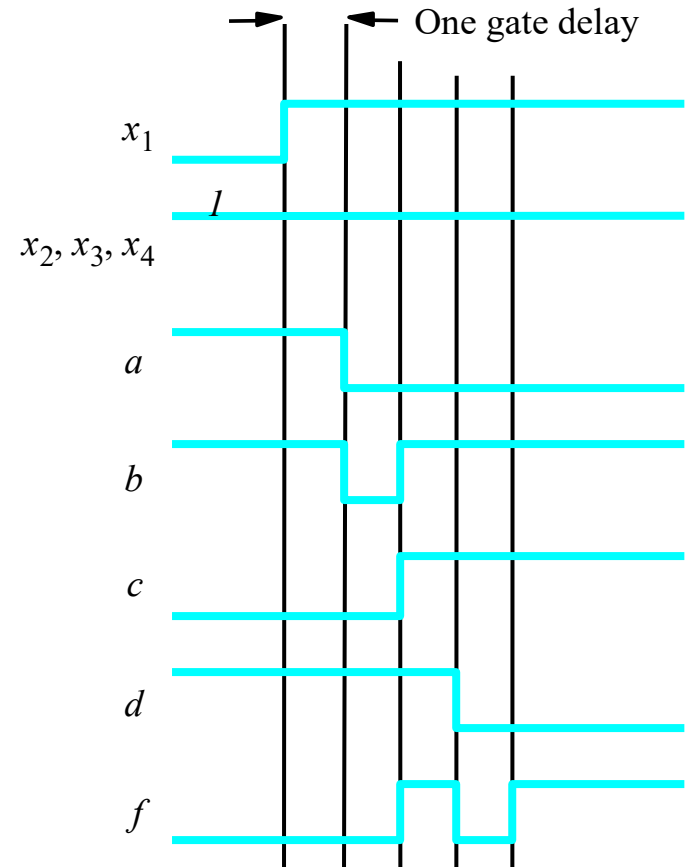
Dynamic hazards (in multi-level circuits)

Circuit with a dynamic hazard and its timing diagram are shown below



(a) Circuit

- Dynamic hazards are mainly multi-level circuits and not easy to deal with
- We can try to avoid such hazards simply by using 2-level circuits and ensuring that there is no static hazards



(b) Timing diagram

Remarks

- The circuit that generates the next-state variables in an ASC must be hazard free. A glitch can cause the circuit to enter an incorrect state and possibly become stable in that state.
- In SSC, the input signals must be stable within the setup and hold times of FFs. Glitches outside the setup and hold times of FFs do not matter much for next state
- For pure combinational circuits (without feedback), we do not worry too much about glitches
- But glitches in general should be prevented. They also waste power.